



Algoritmos e Programação

Prof. Me. Fiorin
andre.fiorin@iffarroupilha.edu.br



Recapitulando

- Na última aula falamos sobre...
 - Modularização
 - Procedimentos
 - Funções



Aula 10

Manipulação de Strings



Manipulação de Strings

- Na programação, uma string é um conjunto de caracteres utilizado para representar uma palavra, frase ou texto.
- Ao contrário de outras linguagens de programação, como Java e C++, a linguagem C não dispõe de um tipo de dado específico para armazenar strings. Portanto, a definição de strings na linguagem C é:

*“Um vetor de caracteres terminado pelo caractere **\0** (nulo)”*

- Exemplo:

p	a	l	a	v	r	a	\0
---	---	---	---	---	---	---	----



Manipulação de Strings

- Sintaxe

```
char <nome> [tamanho];
```

- Exemplos

```
char instituicao[];
```

```
char curso[30];
```

```
char disciplina[20];
```



Manipulação de Strings

- Declaração e inicialização

```
char instituicao[] = {'I', 'F', 'F', 'a', 'r', '\\0'};  
char curso[30] = {'C', 'o', 'm', 'p', 'u', 't', 'a', 'ç', 'ã', 'o', '\\0'};  
char disciplina[20] = {'A', 'l', 'g', 'o', 'r', 'i', 't', 'm', 'o', 's', '\\0'};
```

ou...

```
char instituicao[] = "IFFar";  
char curso[30] = "Computação";  
char disciplina[20] = "Algoritmos";
```



Manipulação de Strings

- Caracteres especiais

Caractere	Uso
\\	Barra
\"	Aspas duplas.
\'	Aspas simples.
\n	Nova linha, com cursor no início.
\r	Retorno ao começo da linha.
\b	Backspace.
\t	Tabulação horizontal (TAB).
\f	Nova linha, com cursor abaixo de onde estava.
\a	Alerta (bipe).
\v	Tabulação vertical.
\nnn ou \xnn	Caractere de valor <i>nnn</i> (octal) ou <i>nn</i> (hexadecimal)



Manipulação de Strings

- Leitura e impressão

- Para fazer a entrada de uma string através do teclado, pode-se utilizar o comando `scanf`.

Exemplo:

```
scanf ("%s" "%s", nome, sobrenome);
```

- A impressão de strings na tela também pode ser realizada com o comando `printf`.

Exemplo:

```
printf ("Resultado: %s %s", nome, sobrenome);
```

- Observação:

- O comando `scanf` captura apenas uma palavra por vez com o formato `“%s”`. Para capturar uma frase, deve-se utilizar a formato `“%[^\n]”`.



Manipulação de Strings

- Como as strings na linguagem C são implementadas através de vetores, a sua manipulação é “complicada”.
 - Necessidade de acessar cada posição do vetor.
- Como operações com strings são bastante comuns (e manipular vetores é uma tarefa cansativa), para facilitar a manipulação de strings, a linguagem C oferece suporte através de funções básicas, implementadas na biblioteca **<string.h>**.

```
#include <string.h>
```



Funções da biblioteca string.h



Manipulação de Strings

- String copy (strcpy)

- Protótipo:

- `char* strcpy (char *destino, char *fonte);`

- Copia o conteúdo da string **fonte** para o vetor **destino**.

- É tarefa do programador garantir que o vetor **destino** seja grande o suficiente para armazenar o conteúdo contido no vetor **fonte**.

- Alternativa ao operador de atribuição ('=').

- Exemplo de aplicação:

- `char nome[10];`
`strcpy (nome, "Oreste");`



Manipulação de Strings

- String length (strlen)

- Protótipo:

- `int strlen (char *palavra);`

- Retorna o tamanho da string palavra.

- Não inclui o caractere nulo no final da string ('\0');

- Exemplo de aplicação:

- `int numCaracteres;`
`numCaracteres = strlen (nome);`
`printf ("O nome contém %d caracteres", numCaracteres);`



Manipulação de Strings

- String compare (strcmp)

- Protótipo:

- `int strcmp (char *string1, char *string2);`

- Compara as duas strings.

- Retorna **zero** se as duas strings forem iguais;
 - Retorna um **número negativo** se string1 vem antes de string2;
 - Retorna um **número positivo** se string2 vem antes de string1.

- Exemplo de aplicação:

- ```
if (strcmp (nome, "Oreste") == 0)
 printf ("As strings são iguais");
```



# Manipulação de Strings

- String concat

- Protótipo:

- `char* strcat (char *string1, char *string2);`

- Concatena (junta) as duas strings.

- Adiciona o conteúdo de string2 ao final do conteúdo de string1.
    - É tarefa do programador garantir que o vetor **destino** seja grande o suficiente para armazenar o conteúdo contido no vetor **fonte**.

- Exemplo de aplicação:

- `char nomeCurso[20] = {'C', 'i', 'ê', 'n', 'c', 'i', 'a', '\0'};`  
`strcat (nomeCurso, " da computação");`



# Manipulação de Strings

- Outras funções

| Protótipo das Funções Básicas                                                  | Funcionalidade                                        |
|--------------------------------------------------------------------------------|-------------------------------------------------------|
| <code>char *strncat(const char *string1, char *string2, size_t n)</code>       | Concatena n caracteres da string2 na string1.         |
| <code>int strncmp(const char *string1, char *string2, size_t n)</code>         | Compara os n primeiros caracteres de duas strings.    |
| <code>char *strncpy(const char *string1, const char *string2, size_t n)</code> | Copia os n primeiros caracteres da string1 na string2 |
| <code>int strcasecmp(const char *s1, const char *s2)</code>                    | Versão case insensitive de <code>strcmp()</code> .    |
| <code>int strncasecmp(const char *s1, const char *s2, int n)</code>            | versão case insensitive de <code>strncmp()</code> .   |



# Manipulação de Strings

- Funções de busca

| Protótipo de Funções de Busca                             | Funcionalidade                                                                |
|-----------------------------------------------------------|-------------------------------------------------------------------------------|
| <code>char *strchr(const char *string, int c)</code>      | Encontra a primeira ocorrência do caracter <code>c</code> na string.          |
| <code>char *strrchr(const char *string, int c)</code>     | Encontra a última ocorrência do caracter <code>c</code> na string.            |
| <code>char *strstr(const char *s1, const char *s2)</code> | Localiza a primeira ocorrência da string <code>s2</code> em <code>s1</code> . |





# Na próxima aula

- Registros (structs)



# Referências

- C++. Disponível em: <https://cplusplus.com/reference/cstring/>